Análisis Arquitectónico y Aplicación Práctica de la Biblioteca jQuery en el Ecosistema Front-End Contemporáneo

La evolución del desarrollo front-end ha sido testigo de una transformación radical desde los primeros días de la web estática hacia las aplicaciones altamente interactivas y dinámicas que definen la experiencia del usuario moderno. En el centro de esta metamorfosis se encuentra la biblioteca jQuery, una herramienta que, desde su lanzamiento en 2006, no solo simplificó la interacción con el Document Object Model (DOM), sino que estableció las bases de cómo los desarrolladores abordan la compatibilidad entre navegadores y la manipulación de eventos.[1] A pesar del surgimiento de marcos de trabajo reactivos modernos, la relevancia de jQuery persiste en 2026, impulsada por una base instalada masiva y la reciente actualización a la versión 4.0, que alinea la biblioteca con los estándares contemporáneos de JavaScript.[3]

Aspectos Fundamentales y el Rol de las Bibliotecas JavaScript en el Desarrollo Front-End

El desarrollo front-end se basa en el equilibrio entre el rendimiento nativo y la velocidad de desarrollo. En este contexto, una biblioteca JavaScript actúa como una capa de abstracción que encapsula tareas complejas y repetitivas en funciones de alto nivel.[4] Mientras que Vanilla JavaScript se refiere al uso del lenguaje en su forma pura, sin dependencias externas, el uso de bibliotecas como jQuery permite a los desarrolladores evitar lo que se ha descrito como un "trabajo de Sísifo" al enfrentarse a la creciente complejidad de las aplicaciones web.[4]

El rol principal de una biblioteca es estandarizar el comportamiento del código en diferentes entornos de ejecución. Históricamente, los navegadores implementaban las APIs del DOM de manera inconsistente, lo que obligaba a los desarrolladores a escribir múltiples versiones del mismo código para asegurar la funcionalidad en Internet Explorer, Firefox y Safari.[2] jQuery resolvió este problema mediante una interfaz unificada, permitiendo que un único comando funcionara de manera predecible en todos los navegadores principales.[1]

La decisión de utilizar una biblioteca frente a Vanilla JS o un marco de trabajo (framework) depende de factores críticos como la complejidad del proyecto, el tamaño del equipo y los requisitos de rendimiento. Los marcos de trabajo modernos como React o Vue introducen paradigmas de gestión de estado y arquitecturas basadas en componentes que son ideales para aplicaciones de gran escala, pero que conllevan una curva de aprendizaje pronunciada y un gasto general de rendimiento que puede ser innecesario para sitios web de menor envergadura o prototipos rápidos.[6]

Característica	Vanilla JavaScript	Bibliotecas (jQuery)	Marcos de Trabajo (React, Vue, Angular)
Abstracción	Nula (Acceso directo a Web APIs)	Media (Funciones simplificadas)	Alta (Arquitectura de componentes)
Rendimiento	Máximo (Sin sobrecarga)	Moderado (Peso del archivo JS)	Variable (Depende del Virtual DOM y runtime)
Curva de Aprendizaje	Alta (Requiere dominio del lenguaje)	Baja (Sintaxis intuitiva)	Media-Alta (Nuevos paradigmas)
Mantenibilidad	Difícil en proyectos grandes	Moderada (Requiere disciplina)	Alta (Patrones estandarizados)
Uso Ideal	Micro-servicios, componentes críticos	Sitios web, prototipos, legacy	Single Page Applications (SPA) complejas

El impacto de las bibliotecas se extiende más allá de la mera escritura de código; fomentan la creación de ecosistemas donde los desarrolladores pueden compartir soluciones a problemas comunes. jQuery, en particular, impulsó el desarrollo de miles de complementos que permitieron a programadores con conocimientos básicos implementar galerías de imágenes, validaciones de formularios y efectos visuales complejos que de otro modo habrían requerido una experiencia avanzada en ingeniería de software.[1]

A medida que avanzamos hacia 2026, el rol de jQuery ha evolucionado. Ya no se considera la herramienta por defecto para construir la lógica de negocio de aplicaciones complejas, pero sigue siendo el estándar de oro para la manipulación directa del DOM en sistemas de gestión de contenidos como WordPress y en entornos donde la estabilidad y la compatibilidad con navegadores antiguos o especializados son primordiales.[2]

Adquisición e Integración de la Biblioteca en

Proyectos Web

La incorporación de jQuery en un documento HTML es el primer paso crítico para acceder a sus capacidades dinámicas. Existen dos metodologías principales para la obtención y uso de la biblioteca: la descarga local y el uso de Redes de Entrega de Contenido (CDN).[9]

La descarga local implica obtener el archivo comprimido (minificado) desde el sitio oficial de jQuery y alojarlo en el servidor propio. Este enfoque garantiza que la aplicación sea autosuficiente y funcione en entornos sin conexión a Internet o en intranets corporativas con restricciones de seguridad.[11] Por otro lado, la integración mediante CDN, utilizando servidores de proveedores como Google, Microsoft o Cloudflare, ofrece ventajas significativas en términos de velocidad de carga y ahorro de ancho de banda.[9]

Una de las ventajas más citadas del uso de CDN es el almacenamiento en caché del navegador. Si un usuario ha visitado previamente un sitio web que utiliza la misma versión de jQuery desde el mismo CDN, el archivo ya estará almacenado localmente en su dispositivo, eliminando la necesidad de una nueva descarga.[9] Además, los CDNs distribuyen el contenido desde el servidor más cercano geográficamente al usuario, reduciendo la latencia.[13]

Método de Inclusión	**Ventajas**	**Desventajas**
Archivo Local	Control total del archivo, funciona offline, mayor privacidad.	Consume ancho de banda del servidor, carga inicial más lenta.
CDN (Público)	Mayor velocidad por cercanía (geolocalización), caché compartido, ahorro de recursos.	Dependencia de terceros, riesgo de bloqueo por firewalls.
Híbrido (CDN + Fallback)	Combina velocidad con fiabilidad ante fallos del servidor externo.	Requiere un pequeño fragmento de código adicional.

Para mitigar el riesgo de que un CDN no esté disponible, los expertos recomiendan implementar un mecanismo de "fallback" o respaldo. Este script intenta cargar jQuery desde un

CDN y, si la variable global de jQuery no se detecta, procede a cargar la copia local.[9]

Con el lanzamiento de jQuery 4.0, la biblioteca ha adoptado los módulos de ECMAScript (ESM), lo que permite a los desarrolladores modernos utilizar la instrucción import para integrar jQuery de manera más limpia en flujos de trabajo basados en herramientas como Webpack o Vite.[3] Esta actualización representa un esfuerzo por reducir el peso de la biblioteca al permitir que solo se carguen las partes necesarias, mejorando así el rendimiento del sitio web.[3]

El Document Object Model y la Filosofía de Selección de jQuery

Para comprender cómo jQuery incorpora elementos dinámicos, es esencial entender el Document Object Model (DOM). El DOM es una representación estructurada en forma de árbol de los elementos de un documento HTML, donde cada etiqueta, atributo y fragmento de texto es un nodo que puede ser manipulado mediante programación.[14] jQuery actúa como un puente que simplifica la navegación por este árbol, permitiendo a los desarrolladores localizar y modificar elementos con una precisión quirúrgica.[17]

La función central de jQuery es el selector $(), que utiliza la sintaxis de los selectores CSS para identificar elementos en la página.[5] Esta elección de diseño fue revolucionaria en su momento, ya que permitió a los diseñadores web con conocimientos de CSS empezar a programar comportamientos dinámicos sin una curva de aprendizaje técnica profunda.[1]

Categorías de Selectores y Precisión Técnica

La capacidad de selección de jQuery se extiende más allá de los selectores básicos de ID y clase. La biblioteca incluye un motor de selección llamado Sizzle que permite filtrados complejos basados en el estado del elemento, su posición en el DOM o sus atributos.[5]

- **Selectores Jerárquicos**: Permiten seleccionar elementos basados en su relación con otros nodos, como hijos directos (parent > child), descendientes (ancestor descendant) o hermanos adyacentes (prev + next).[5]
- **Filtros de Contenido**: jQuery puede localizar elementos que contienen un texto específico (:contains()) o que tienen ciertos hijos (:has()), lo cual es fundamental para manipular interfaces generadas dinámicamente donde no siempre se conocen los IDs de antemano.[5]
- **Filtros de Visibilidad y Estado**: Los selectores :hidden y :visible, así como los estados de formulario como :checked o :disabled, permiten crear lógica de interfaz que responde al estado actual del usuario sin necesidad de variables de estado complejas.[5]

La manipulación básica de objetos en el DOM comienza una vez que el elemento ha sido seleccionado. jQuery devuelve un objeto que contiene una colección de elementos coincidentes, sobre los cuales se pueden encadenar métodos para realizar múltiples acciones en una sola línea de código.[8] Este concepto de "encadenamiento" (chaining) no solo reduce la

cantidad de código escrito, sino que mejora la legibilidad al mostrar claramente la secuencia de transformaciones aplicadas a un elemento.[8]

Manipulación Dinámica del DOM: De la Estructura al Estilo

La manipulación del DOM con jQuery se divide en tres áreas principales: gestión de contenido, modificación de atributos y control de estilos CSS. Estas herramientas permiten que un documento HTML estático se convierta en una interfaz viva que responde a los datos y a las acciones del usuario.[8]

Gestión de Contenido y Estructura

Los métodos .html() y .text() son los pilares de la actualización de contenido. Mientras que .html() permite insertar etiquetas HTML completas, lo que facilita la creación de estructuras complejas sobre la marcha, .text() se utiliza para insertar contenido plano, lo cual es una práctica recomendada de seguridad para evitar ataques de Cross-Site Scripting (XSS) al manejar datos introducidos por el usuario.[20]

Para la alteración estructural, jQuery ofrece métodos de inserción que permiten añadir elementos en posiciones específicas respecto a otros nodos:

- **Inserción Interna**: .append() y .prepend() añaden contenido al final o al principio de un elemento, respectivamente.[16]
- **Inserción Externa**: .after() y .before() colocan el nuevo contenido inmediatamente después o antes del elemento seleccionado.[16]
- **Eliminación**: .remove() elimina el elemento y sus descendientes del DOM, mientras que .empty() solo elimina el contenido interno, dejando el contenedor intacto.[16]

Control de Atributos y Propiedades

La manipulación de atributos mediante .attr() y .removeAttr() permite cambiar dinámicamente imágenes (src), enlaces (href) o cualquier otro metadato de una etiqueta.[20] Con el avance de JavaScript, jQuery introdujo el método .prop() para manejar propiedades booleanas del DOM, como el estado de un checkbox o el enfoque de un input, diferenciándolas de los atributos HTML iniciales.[17]

Dinamismo Visual mediante CSS

jQuery permite una interacción fluida con las hojas de estilo mediante el método .css(), que puede tanto consultar como establecer propiedades de estilo.[18] Sin embargo, la arquitectura moderna sugiere que la lógica de presentación debe residir en clases CSS predefinidas. jQuery facilita esto mediante los métodos .addClass(), .removeClass() y .toggleClass(), permitiendo que el JavaScript actúe como un interruptor que activa o desactiva estilos complejos definidos

en el archivo CSS.[5]

Método de Estilo	Función Principal	Caso de Uso Típico
.css()	Establece o recupera estilos inline.	Cálculos dinámicos de posición o tamaño.
.addClass()	Añade una o más clases a un elemento.	Activar un estado visual (ej. "seleccionado").
.removeClass()	Elimina clases de un elemento.	Limpiar estados de error o selección.
.toggleClass()	Alterna la presencia de una clase.	Menús desplegables o interruptores de modo oscuro.

Manejo de Eventos: El Motor de la Interactividad

El manejo de eventos es el mecanismo por el cual jQuery detecta las acciones del usuario, como clics, movimientos del ratón o pulsaciones de teclas, y ejecuta código en respuesta.[22] El sistema de eventos de jQuery es uno de sus componentes más robustos, ya que unifica las diferentes formas en que los navegadores gestionan la propagación y captura de eventos.[15]

Tipos de Eventos y Sintaxis Básica

La biblioteca ofrece métodos abreviados para los eventos más comunes, como .click(), .dblclick(), .mouseenter() y .mouseleave().[22] Estos métodos son, en realidad, atajos para el método .on(), que es la forma recomendada y más versátil de vincular controladores de eventos desde la versión 1.7 de jQuery.[24]

El método .on() permite:

1. Vincular múltiples eventos al mismo controlador.[24]
2. Pasar datos personalizados a la función que maneja el evento.[24]
3. Implementar la delegación de eventos, un concepto crucial para aplicaciones

dinámicas.[24]

Delegación de Eventos para Elementos Dinámicos

Un problema común en el desarrollo web ocurre cuando se añaden elementos al DOM después de que la página se ha cargado (por ejemplo, mediante una petición AJAX o una acción del usuario). Si se utiliza un vínculo de evento directo como $('.btn').click(), los nuevos botones creados dinámicamente no responderán al clic, ya que el controlador solo se vinculó a los elementos que existían en el momento de la ejecución del script.[24]

La delegación de eventos resuelve esto vinculando el controlador a un elemento padre "estable" (que no se elimina de la página) y especificando un selector hijo. Debido al fenómeno de "bubbling" o burbujeo, los eventos ocurridos en el hijo se propagan hacia arriba en el DOM, donde el padre los captura y verifica si coinciden con el selector indicado.[26]

Sintaxis de Delegación:

$("#contenedor-padre").on("click", ".boton-dinamico", function() {... });

Esta técnica no solo asegura que los nuevos elementos sean funcionales, sino que también mejora el rendimiento de la aplicación al reducir el número de controladores de eventos individuales en la memoria.[25]

Categoría de Evento	Ejemplos de Métodos	Acción Desencadenante
Ratón	click, dblclick, hover, mousedown, mouseup	Interacción física con el dispositivo apuntador.
Teclado	keydown, keypress, keyup	Entrada de datos mediante el teclado.
Formulario	submit, change, focus, blur	Ciclo de vida de la entrada de datos del usuario.
Documento	ready, scroll, resize, unload	Cambios en el estado global de la ventana o documento.

Integración de Bootstrap y jQuery: Sinergia de Frameworks

Bootstrap, el framework de diseño más utilizado en el mundo, dependió históricamente de jQuery para potenciar sus componentes interactivos como modales, carousels y menús desplegables.[29] Aunque Bootstrap 5 ha eliminado la dependencia obligatoria de jQuery en favor de Vanilla JavaScript, mantiene una capa de compatibilidad que permite a los desarrolladores seguir utilizando la sintaxis de jQuery para controlar sus componentes.[31]

El Rol de jQuery en Bootstrap 4 y 5

En versiones como Bootstrap 4, jQuery era un requisito estricto. Los plugins de JavaScript de Bootstrap se registran como extensiones de jQuery, lo que significa que se pueden invocar mediante métodos como $('#miModal').modal('show').[29] Además de jQuery, componentes como los tooltips y popovers requieren la biblioteca Popper para la gestión inteligente del posicionamiento.[30]

Para proyectos que migran a Bootstrap 5, la biblioteca detecta automáticamente si jQuery está presente en el objeto window y, de ser así, añade todos sus componentes al sistema de complementos de jQuery. Esto permite una transición suave para equipos que prefieren la ergonomía de jQuery frente a la sintaxis más verbosa de addEventListener nativo.[31]

Inicialización y Control Programático

A diferencia de los estilos CSS de Bootstrap que funcionan simplemente añadiendo clases, muchos de sus componentes de JavaScript son "opt-in" por razones de rendimiento. Esto significa que deben ser inicializados manualmente mediante código.[30] Un ejemplo común es la activación de todos los tooltips de una página:

JavaScript

```
$(function () {
  $('[data-toggle="tooltip"]').tooltip();
});
```

Este fragmento demuestra el uso de un selector de atributo para encontrar todos los elementos destinados a ser tooltips y aplicarles el constructor de Bootstrap en una sola operación colectiva.[30]

Plugins Bootstrap-jQuery: Extendiendo las Capacidades de la Web

Un plugin de jQuery es un fragmento de código escrito por terceros que extiende el prototipo de la biblioteca para ofrecer funcionalidades adicionales.[5] El uso de plugins es una estrategia fundamental cuando se necesita resolver problemas complejos que el núcleo de jQuery o Bootstrap no cubren, como calendarios avanzados, editores de texto enriquecido o rejillas de datos interactivas.[36]

¿Qué es y cuándo usar un Plugin?

Un desarrollador debe considerar el uso de un plugin cuando el costo de construir una funcionalidad desde cero supera los beneficios de personalización. Los plugins ofrecen soluciones probadas que ya manejan casos de borde, accesibilidad y compatibilidad entre navegadores.[1]

Para incorporar un plugin bootstrap-jQuery, se debe seguir un orden estricto de carga de scripts para evitar errores de dependencia:

1. Biblioteca jQuery (El núcleo).
2. Popper.js (Si el plugin o Bootstrap lo requieren para posicionamiento).
3. Bootstrap JS (El framework base).
4. El archivo del Plugin de Terceros (Que depende de los anteriores).
5. Código de inicialización personalizado.[30]

Ejemplos de Plugins Comunes y su Aplicación

El ecosistema de plugins es vasto y cubre casi cualquier necesidad de interfaz de usuario. Algunos de los más destacados en entornos profesionales incluyen:

- **DataTables**: Posiblemente el plugin más potente para la gestión de tablas HTML. Permite añadir paginación, búsqueda en tiempo real, ordenación de columnas y exportación de datos a PDF o Excel con una configuración mínima. Se integra perfectamente con Bootstrap para mantener la estética del sitio.[39]
- **SweetAlert2**: Reemplaza los cuadros de diálogo alert y confirm del navegador por modales estilizados, responsivos y altamente configurables que se alinean con los temas de Bootstrap 4 y 5.[42]
- **Slick Carousel**: Un plugin extremadamente flexible para crear carruseles de imágenes y contenido que son totalmente táctiles y responsivos, superando a menudo las capacidades del carrusel nativo de Bootstrap en términos de personalización.[36]
- **Select2**: Transforma los elementos <select> estándar en cuadros de selección avanzados con soporte para búsqueda, carga remota de datos mediante AJAX y selección múltiple estilizada.[37]
- **VvvebJs**: Un ejemplo de plugin de gran escala que actúa como un constructor de sitios

web de arrastrar y soltar, permitiendo a los usuarios finales crear layouts utilizando componentes de Bootstrap directamente en el navegador.[37]

Plugin	Categoría	Funcionalidad Clave
DataTables	Datos	Ordenación, búsqueda y paginación de tablas complejas.
SweetAlert2	UI / Modales	Diálogos de confirmación y alertas estéticas.
Slick	Multimedia	Sliders y carruseles táctiles de alto rendimiento.
Select2	Formularios	Selectores con búsqueda y soporte AJAX.
jQuery Validation	Formularios	Validación robusta de entradas de usuario en el lado del cliente.

La personalización de estos plugins se realiza generalmente pasando un objeto de opciones al inicializarlos. Por ejemplo, al usar un modal de SweetAlert2, se pueden definir los colores de los botones, el icono de estado y si el usuario puede cerrar el diálogo haciendo clic fuera de él.[44]

El Futuro de jQuery: 2026 y Más Allá

A medida que nos adentramos en la segunda mitad de la década, surge la pregunta de si jQuery sigue siendo una habilidad necesaria para el desarrollador front-end moderno. La respuesta es un rotundo sí, aunque con matices estratégicos.

El Renacimiento con jQuery 4.0

La llegada de jQuery 4.0 ha revitalizado la biblioteca al eliminar el lastre de navegadores

obsoletos como Internet Explorer 10 y adoptar estándares de seguridad modernos como Trusted Types, que ayudan a prevenir inyecciones de código malicioso.[3] La migración a módulos ESM permite que jQuery se integre en arquitecturas modernas de micro-frontends y aplicaciones desacopladas, manteniendo su promesa de simplicidad.[3]

Conclusiones y Recomendaciones Profesionales

El análisis de la trayectoria de jQuery revela que su mayor éxito no fue solo su código, sino la democratización del desarrollo web. Al permitir que millones de personas interactuaran con el DOM de manera intuitiva, jQuery aceleró la evolución de la web.[1]

Para un desarrollador contemporáneo, el conocimiento de jQuery es indispensable por tres razones fundamentales:

1. **Mantenimiento de Sistemas Críticos**: Gran parte de la web actual, incluyendo infraestructuras de comercio electrónico y portales gubernamentales, se basa en jQuery y Bootstrap 4.[2]
2. **Productividad en Prototipado**: Para pruebas de concepto y herramientas internas, la velocidad que ofrece jQuery para manipular el DOM y realizar peticiones AJAX sigue siendo superior a la configuración inicial de marcos de trabajo más pesados.[1]
3. **Comprensión de los Fundamentos**: Aprender cómo jQuery abstrae el DOM ayuda a entender cómo funcionan las APIs nativas del navegador, lo que convierte al programador en un desarrollador de Vanilla JavaScript más valioso y capaz.[4]

En conclusión, jQuery no es una reliquia del pasado, sino una herramienta madura, estable y ahora modernizada que continúa cumpliendo su rol esencial de facilitar la interactividad web. Su integración con Bootstrap y su vasto ecosistema de plugins aseguran que seguirá siendo una pieza clave del rompecabezas del desarrollo front-end durante los próximos años.[2] La clave para el éxito profesional reside en saber cuándo utilizar la potencia nativa de Vanilla JS y cuándo apoyarse en la robustez y simplicidad de jQuery para "escribir menos y hacer más".[1]

Obras citadas

1. jQuery en 2025: ¿Por qué fue amado y por qué hoy algunos lo evitan?, fecha de acceso: abril 3, 2026, https://smilecomunicacion.com/web/jquery-en-2025/
2. Why I Still Use jQuery in 2025 (and When Not to) - Docker, fecha de acceso: abril 3, 2026, https://www.docker.com/blog/why-i-still-use-jquery-2025/
3. jQuery 4.0 ya está aquí: por qué sigue vivo y qué significa para tu sitio web en 2026, fecha de acceso: abril 3, 2026, https://blog.clickpanda.com/web/jquery-4-0-esta-aqui/
4. Vanilla JS vs JavaScript vs Frameworks, and When to Use Them - Webix Blog, fecha de acceso: abril 3, 2026, https://blog.webix.com/vanillajs-vs-framework-when-using-js-libraries/
5. jQuery, integración de contenido interactivo - Eniun, fecha de acceso: abril 3, 2026, https://www.eniun.com/jquery-integracion-contenido-interactivo/

6. The Great Debate: Vanilla JS vs Modern Frameworks | by Rahul Nagubandi - Medium, fecha de acceso: abril 3, 2026, https://medium.com/@rahulnagubandi/the-great-debate-vanilla-js-vs-modern-frameworks-0768ff83c547
7. Modern Frontend frameworks vs VanillaJS - Reddit, fecha de acceso: abril 3, 2026, https://www.reddit.com/r/Frontend/comments/18wudcw/modern_frontend_frameworks_vs_vanillajs/
8. 29. jQuery -DOM Manipulation (Beginner's Guide)..... | by Muneezabilal - Medium, fecha de acceso: abril 3, 2026, https://medium.com/@muneezabilal0077/29-jquery-dom-manipulation-beginners-guide-9488849253bb
9. ¿Qué es jQuery CDN y cómo usarlo en tu sitio web? - Inesem, fecha de acceso: abril 3, 2026, https://www.inesem.es/revistadigital/informatica-y-tics/jquery-cdn
10. Biblioteca jQuery: ¿Qué es y Cómo Conectarla? - CodigoNautas, fecha de acceso: abril 3, 2026, https://codigonautas.com/jquery-que-es-como-conectarla/
11. ¿Cúal es la diferencia entre poner un link CDN y descargar? - Stack Overflow en español, fecha de acceso: abril 3, 2026, https://es.stackoverflow.com/questions/15432/c%C3%BAal-es-la-diferencia-entre-poner-un-link-cdn-y-descargar
12. jQuery CDN o hosting local de las librerías - Desarrollo Web, fecha de acceso: abril 3, 2026, https://desarrolloweb.com/articulos/jquery-cdn-hosting-local.html
13. 5 Reasons to Use jQuery CDN in Your Website - GlobalDots, fecha de acceso: abril 3, 2026, https://www.globaldots.com/resources/blog/5-reasons-to-use-jquery-cdn-in-your-website/
14. fecha de acceso: abril 3, 2026, https://www.godaddy.com/resources/latam/desarrollo/jquery-que-es-documentos-html#:~:text=jQuery%20permite%20manipular%20el%20DOM,visuales%20como%20desvanecimientos%20o%20deslizamientos.
15. jQuery: tutorial sobre selectores, sintaxis y mucho más - IONOS, fecha de acceso: abril 3, 2026, https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/jquery-tutorial-de-integracion-y-primeros-pasos/
16. DOM Manipulation with jQuery - Diginode, fecha de acceso: abril 3, 2026, https://diginode.in/javascript/dom-manipulation-with-jquery/
17. jQuery Tutorial - GeeksforGeeks, fecha de acceso: abril 3, 2026, https://www.geeksforgeeks.org/jquery/jquery-tutorial/
18. jQuery DOM Manipulation: Complete On-Demand Guide, fecha de acceso: abril 3, 2026, https://www.guvi.in/blog/jquery-dom-manipulation/
19. jQuery API Documentation, fecha de acceso: abril 3, 2026, https://api.jquery.com/
20. Modificar elementos con jQuery | Modificación del DOM con jQuery | HTML/JS: hacer páginas web interactivas con jQuery | Programación de computadoras - Khan Academy, fecha de acceso: abril 3, 2026, https://es.khanacademy.org/computing/computer-programming/html-js-jquery/d

om-modification-with-jquery/pt/modifying-elements-with-jquery
21. Utilizando jQuery para manipular el DOM - Somos PNT, fecha de acceso: abril 3, 2026, https://somospnt.com/blog/267-utilizando-jquery-para-manipular-el-dom
22. jQuery Events - GeeksforGeeks, fecha de acceso: abril 3, 2026, https://www.geeksforgeeks.org/jquery/jquery-events/
23. Eventos · Desarrollo de aplicaciones web - juanda, fecha de acceso: abril 3, 2026, https://juanda.gitbooks.io/webapps/content/jQuery/eventos.html
24. jQuery Event Basics | jQuery Learning Center, fecha de acceso: abril 3, 2026, https://learn.jquery.com/events/event-basics/
25. .on() | jQuery API Documentation, fecha de acceso: abril 3, 2026, https://api.jquery.com/on/
26. Delegate jQuery event handlers when working with dynamic content - Phil Kurth, fecha de acceso: abril 3, 2026, https://philkurth.com.au/delegate-jquery-event-handlers-when-working-with-dynamic-content/
27. How TO Make JQuery Aware Of Id Elements In Dynamically Loaded HTML? - TutorialsPoint, fecha de acceso: abril 3, 2026, https://www.tutorialspoint.com/article/how-to-make-jquery-aware-of-id-elements-in-dynamically-loaded-html
28. Event binding on dynamically created elements? - Stack Overflow, fecha de acceso: abril 3, 2026, https://stackoverflow.com/questions/203198/event-binding-on-dynamically-created-elements
29. Tutorial Bootstrap: primeros pasos con el framework de Twitter - IONOS, fecha de acceso: abril 3, 2026, https://www.ionos.com/es-us/digitalguide/paginas-web/desarrollo-web/tutorial-bootstrap-los-primeros-pasos/
30. Introduction · Bootstrap v4.6, fecha de acceso: abril 3, 2026, https://getbootstrap.com/docs/4.6/getting-started/introduction/#js
31. ¿Por qué Bootstrap 5 dejo de usar jQuery? y ¿Qué es lo que viene? - Platzi, fecha de acceso: abril 3, 2026, https://platzi.com/blog/por-que-bootstrap-5-dejo-de-usar-jquery-y-que-es-lo-que-viene/
32. JavaScript · Bootstrap v5.0, fecha de acceso: abril 3, 2026, https://getbootstrap.com/docs/5.0/getting-started/javascript/
33. JavaScript - Bootstrap, fecha de acceso: abril 3, 2026, https://getbootstrap.com/docs/4.3/getting-started/javascript/
34. Top 5 Bootstrap Plugins to Know in 2025 - Developers Zone, fecha de acceso: abril 3, 2026, https://www.developers-zone.com/top-5-bootstrap-plugins-2025/
35. How to Customize Bootstrap jQuery Plugins ? - GeeksforGeeks, fecha de acceso: abril 3, 2026, https://www.geeksforgeeks.org/jquery/how-to-customize-bootstrap-jquery-plugins/
36. A curated list of awesome jQuery plugins, resources and other shiny things. - GitHub, fecha de acceso: abril 3, 2026, https://github.com/petk/awesome-jquery

37. 250+ Best jQuery Bootstrap Tutorial & Plugins with example, fecha de acceso: abril 3, 2026, https://www.bestjquery.com/example/bootstrap/
38. Web SPA - 03 - Bootstrap, jQuery, Popper - YouTube, fecha de acceso: abril 3, 2026, https://www.youtube.com/watch?v=Vsb4v9g6r5c
39. DataTables example - Bootstrap 4, fecha de acceso: abril 3, 2026, https://editor.datatables.net/examples/datatables/bootstrap4.html
40. DataTables example - Bootstrap 4, fecha de acceso: abril 3, 2026, https://editor.datatables.net/examples/styling/bootstrap4
41. DataTables example - Bootstrap 4, fecha de acceso: abril 3, 2026, https://www.misin.msu.edu/0/js/DataTablesP/DataTables-1.10.18/examples/styling/bootstrap4.html
42. SweetAlert2 - a beautiful, responsive, customizable and accessible (WAI-ARIA) replacement for JavaScript's popup boxes, fecha de acceso: abril 3, 2026, https://sweetalert2.github.io/
43. Using sweetalert2 with bootstrap 4 - javascript - Stack Overflow, fecha de acceso: abril 3, 2026, https://stackoverflow.com/questions/42287895/using-sweetalert2-with-bootstrap-4
44. SweetAlert2 - a beautiful, responsive, customizable and accessible (WAI-ARIA) replacement for JavaScript's popup boxes, fecha de acceso: abril 3, 2026, https://www.apdperu.org/wp-content/themes/apd/vendor/sweetalert2/
45. 20 Best jQuery Plugins & Effects for Websites (Still Working in 2026) - Colorlib, fecha de acceso: abril 3, 2026, https://colorlib.com/wp/jquery-plugins-effects/
46. SweetAlert2 - supported fork of t4t5/sweetalert - SSO UPN Jatim, fecha de acceso: abril 3, 2026, https://sso.upnjatim.ac.id/assets/js/plugin/sweetalert/
47. Vanilla JS or framework/library for beginner? : r/learnprogramming - Reddit, fecha de acceso: abril 3, 2026, https://www.reddit.com/r/learnprogramming/comments/wgm8xv/vanilla_js_or_frameworklibrary_for_beginner/